

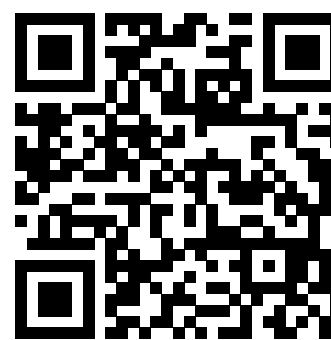
oauth2-passkey

Passwordless Authentication Library for Rust

Kimitoshi Takahashi (@ktaka) | Tokyo Rust Show & Tell | 2026/03/31

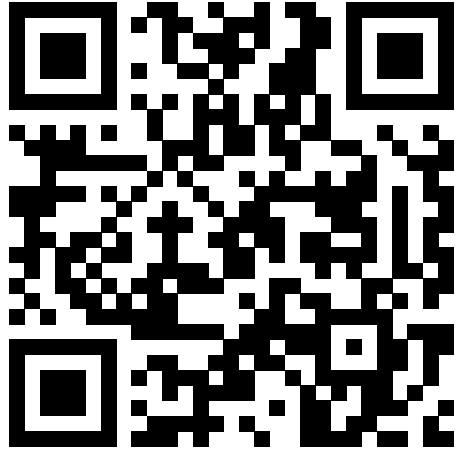


GitHub



Author

Try it Yourself!



passkey-demo.ccmp.jp

Try passwordless auth on your own device:

1. **Login with Google**
2. **Register your fingerprint**
(Passkey Promotion — the library prompts you)
3. **Logout → Login with fingerprint only**
No redirect, instant
4. **Check account page**
See OAuth2 + Passkey linked to one account

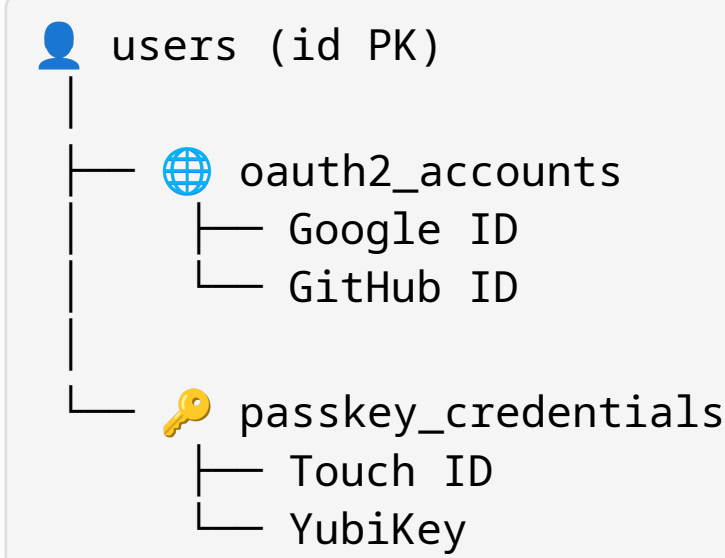
Works with: Google Password Manager, Apple, Windows Hello, YubiKey

Flow: Google Login → Passkey Setup

Step-by-step:

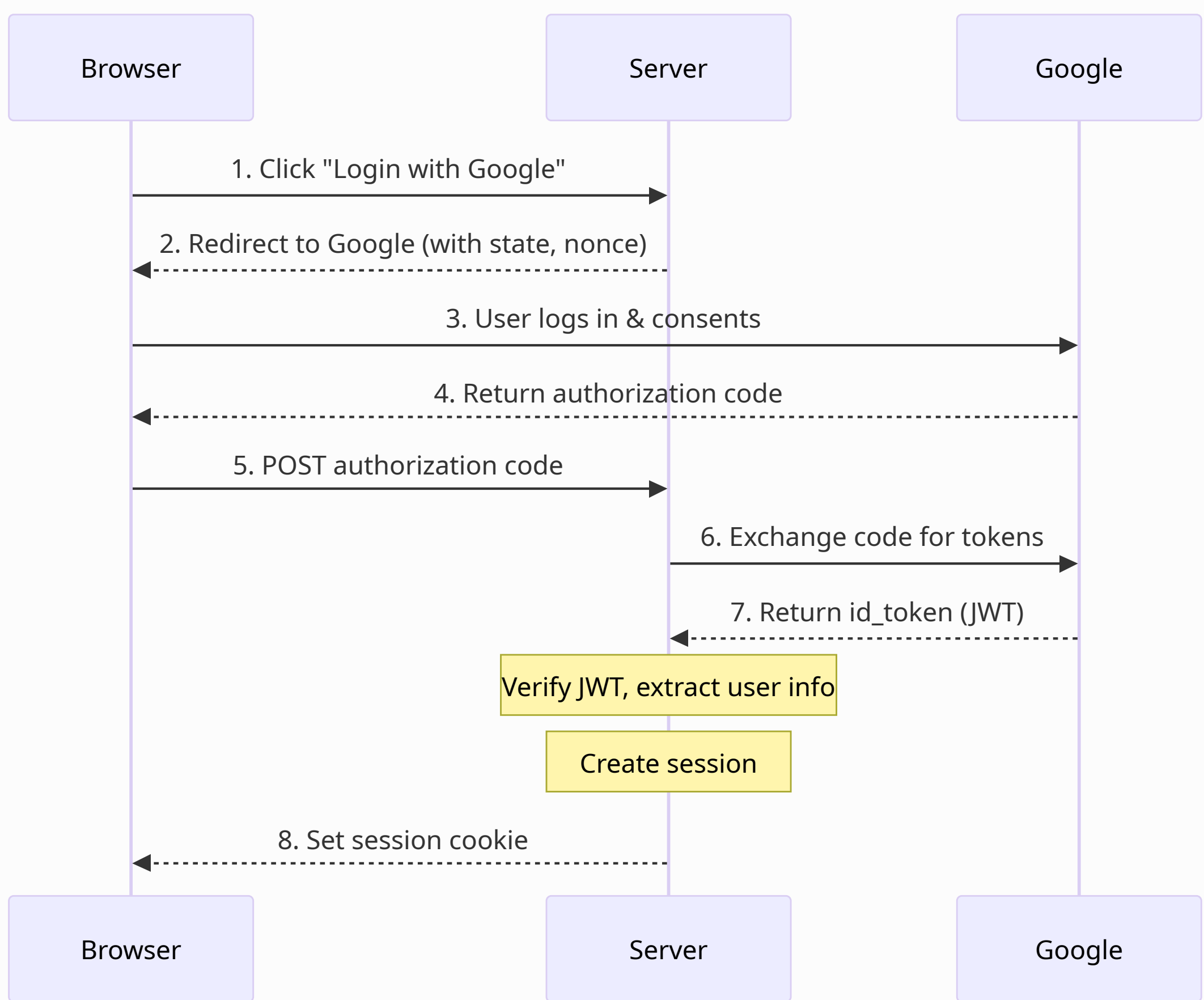
1. **OAuth2 Login**
→ create user linked with Google account
2. **Passkey Promotion**
→ register Passkey → link to user
3. **Next Login**
→ use either Passkey or Google OAuth2

Internal DB structure:



One user can have multiple OAuth2 accounts and multiple Passkey credentials.

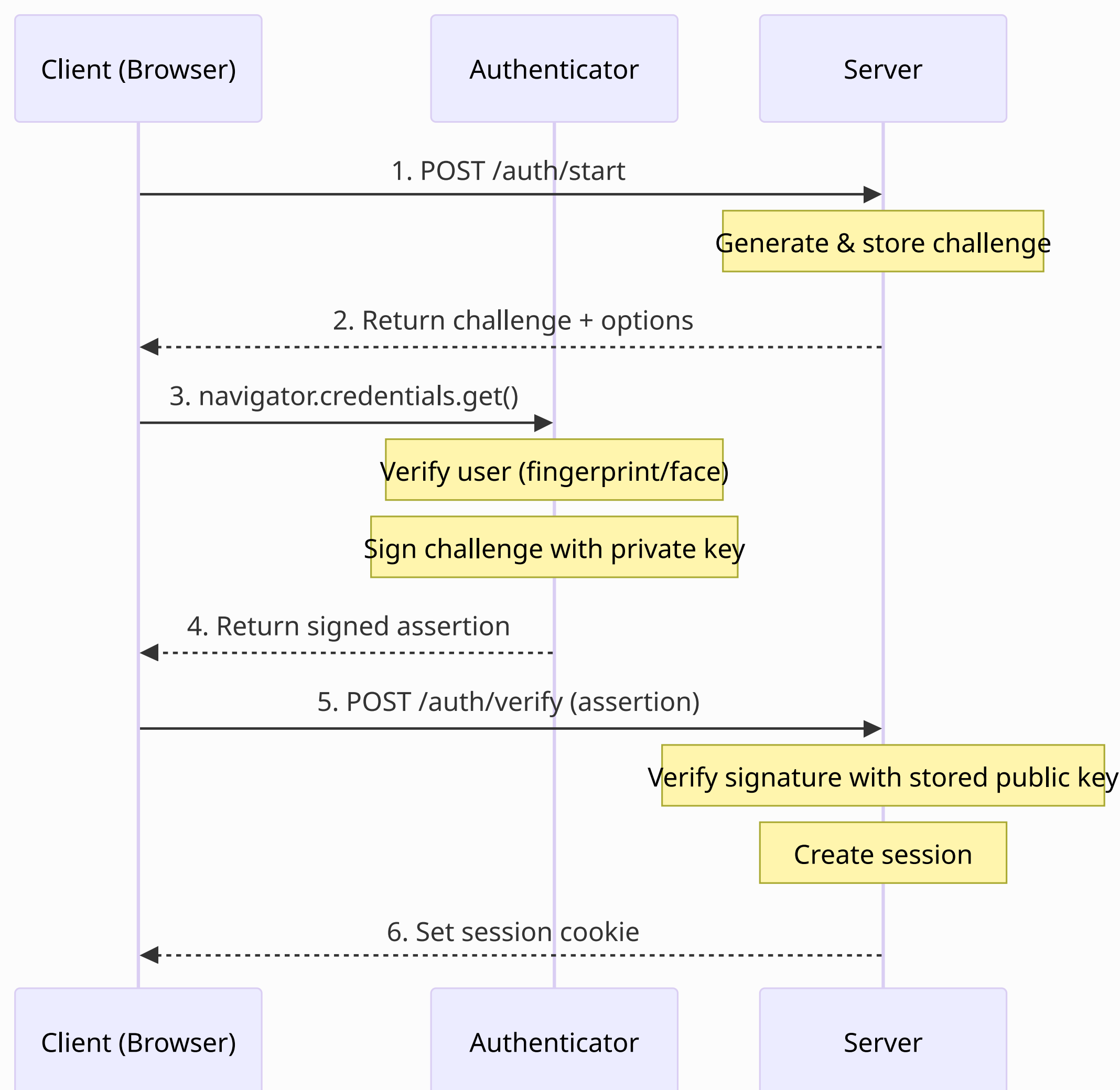
How OAuth2/OIDC Works



Page-redirect based auth:

1. User clicks “Login with Google”
2. Redirect to Google consent screen, get authorization code
3. Server exchanges code for **id_token** (JWT)
4. Extract user info, create session, set cookie

How Passkey/WebAuthn Works



JavaScript-driven, no redirects:

1. Server generates **challenge**
2. Browser calls `navigator.credentials.get()`
3. Authenticator signs challenge with **private key**
4. Server verifies with stored **public key**, set cookie

Authenticators: Google Password Manager, YubiKey, Touch ID, Windows Hello

Usage: Add Auth in 3 Steps

Step 1: Configure `.env` (swap `sqlite` → `postgres/mysql` to change DB):

```
GENERIC_DATA_STORE_TYPE=sqlite
GENERIC_DATA_STORE_URL='sqlite:/tmp/auth.db'
GENERIC_CACHE_STORE_TYPE=memory
OAUTH2_GOOGLE_CLIENT_ID='xxx.apps.googleusercontent.com'
```

Steps 2-3: Initialize and merge router. Built-in login UI, account management, and admin panel included.

```
use oauth2_passkey_axum::{AuthUser, oauth2_passkey_full_router};

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    dotenv().ok();
    oauth2_passkey_axum::init().await?; // 2. Initialize
    let app = Router::new()
        .route("/", get(index))
        .merge(oauth2_passkey_full_router()); // 3. Merge router
    spawn_http_server(3001, app).await?;
    Ok(())
}
```

Page Protection

AuthUser extractor (via `FromRequestParts`):

```
// Required auth - redirects to login if not authenticated
async fn protected(user: AuthUser) -> impl IntoResponse { ... }

// Optional auth - allows anonymous access
async fn public(user: Option<AuthUser>) -> impl IntoResponse { ... }
```

Middleware — choose 401 vs redirect, and skip DB when user info isn't needed:

Middleware	Unauth	DB Query	Extension
<code>is_authenticated_401</code>	401	No	<code>CsrfToken</code>
<code>is_authenticated_redirect</code>	Login	No	<code>CsrfToken</code>
<code>is_authenticated_user_401</code>	401	Yes	<code>AuthUser</code>
<code>is_authenticated_user_redirect</code>	Login	Yes	<code>AuthUser</code>

Storage & LazyLock Pattern

Switch DB by changing `.env` only — no code changes:

```
# Data store: sqlite / postgres / mysql
GENERIC_DATA_STORE_TYPE=sqlite
GENERIC_DATA_STORE_URL='sqlite:/tmp/auth.db'

# Cache store: memory / redis
GENERIC_CACHE_STORE_TYPE=memory
```

SQLite/PostgreSQL/MySQL abstracted via `DataStore` trait, held in a `LazyLock`:

```
static GENERIC_DATA_STORE: LazyLock<Mutex<Box<dyn DataStore>>>
    = LazyLock::new(|| { /* reads env, builds pool */ });

// Dispatch - no runtime downcasting needed
let store = GENERIC_DATA_STORE.lock().await;
match (store.as_sqlite(), store.as_postgres(), store.as_mysql()) {
    (Some(pool), _, _) => do_sqlite(pool).await?,
    (_, Some(pool), _) => do_postgres(pool).await?,
    (_, _, Some(pool)) => do_mysql(pool).await?,
    _ => return Err(...),
}
```

Why LazyLock instead of Axum State?

With Axum State:

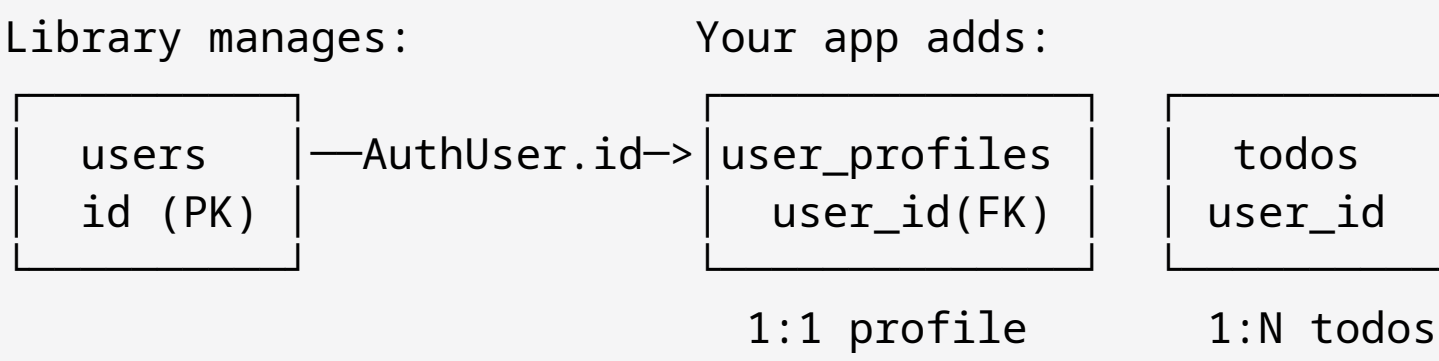
- User must compose `AuthState` into their own `AppState`
- 80+ internal library functions all need `&State`

With LazyLock:

- User just calls `init().await?`
- Any function accesses storage directly
- Fail-fast: bad config panics at startup

Integrating with Your App's Database

Library manages `users`, `oauth2_accounts`, `passkey_credentials`, `sessions`. Your app adds its own tables linked by `AuthUser.id`:



```
async fn create_todo(
    State(state): State<AppState>,
    user: AuthUser, // from oauth2-passkey
    Form(form): Form<TodoForm>,
) -> Result<Response, ...> {
    db::create_todo(&state.pool, &user.id, &form.title).await?;
    Ok(Redirect::to("/").into_response())
}
```

See `demo-profile (1:1)` and `demo-todo (1:N)` for full examples.

Summary

What	OAuth2 + Passkey auth library for Axum
Highlights	Passkey Promotion, Built-in UI, Account Linking
Usage	<code>init()</code> + <code>merge(router)</code> + <code>AuthUser</code> extractor
Protection	Extractor or Middleware (401/redirect, with/without DB)
Storage	SQLite/PostgreSQL/MySQL + Memory/Redis, switch via <code>.env</code>
Your app	Link your data to <code>AuthUser.id</code>

- **crates.io:** `oauth2-passkey`, `oauth2-passkey-axum`
- **GitHub:** `github.com/ktaka-ccmp/oauth2-passkey`
- **Demo:** `passkey-demo.ccmp.jp`